



Ben-Gurion University of the Negev
Faculty of Engineering Sciences
Department of Software and Information systems Engineering

Deep Learning
Assignment 4

The purpose of the assignment

Enabling students to experiment with generative architectures.

Submission instructions:

- a) The assignment due date: 29.06.2026
- b) The assignment is to be carried out using PyTorch.
- c) Submission in **pairs** only. Only **one copy** of the assignment needs to be uploaded to Moodle.
- d) The file name needs to be in the format "ID1_ID2.zip".
- e) Plagiarism of any kind (e.g., GitHub) is forbidden.
- f) Submit your work using a notebook.
- g) The entire project will be submitted as a single zip file, containing both the report (in PDF form only) and the code. It is the students' responsibility to make sure that the file is valid (i.e. can be opened correctly).
- h) The report accompanying the code needs to be a **.docx file** (use Calibri font, text size 12, margins of 2.5cm). The report itself will not exceed **8 pages** of text. Figures (graphs, tables, etc) can be placed an appendix, and must be referenced in the main body of report.

Introduction

Generative models are among the most exciting architecture in the field of deep learning today. Their ability to generate high-quality data with little-to-no interaction or knowledge with the existing data enables us to accomplish tasks thought impossible not long ago. In this assignment, you will implement two generative models on tabular data: a GAN and a Conditional GAN (cGAN).

Instructions

- 1) The dataset for this task is Adult, which is attached to this assignment. The file is in ARFF format, which is a CSV file with a header detailing all possible values (useful for discrete features).
- 2) In your evaluations, use a 80%/20% train-test split (maintain label ratios for the target feature “income”). Report the average results for three experiments, each initialized with a different random seed for train-test sets creation.
- 3) All preprocessing decisions, such as normalization for continuous features or one-hot representations for discrete features, are up to you.
- 4) Training the GAN architecture:
 - a. Use the standard setup taught in class, where the generator receives random noise and produces synthetic tabular data.
 - b. The discriminator component will receive two samples – one real and one synthetic – and attempt to differentiate them. You may use any loss function you wish (within reasonable bounds).
 - c. You have significant freedom in designing your architecture. For example, you may train a separate autoencoder architecture, then use the encoder component to create an embedding of the real sample before it is sent to the discriminator (in this setup, for example, the generator needs to produce the embedding rather than the actual sample). Make sure you justify your design decisions, and make sure to present information that supports them. For example, for the autoencoder scenario described above, provide information about the autoencoder’s training progress.
- 5) Training the cGAN architecture:
 - a. The conditional GAN architecture is very similar to the standard GAN, but the generator receives another “instruction”, specifying what is the desired label of the generated sample. This instruction, usually represented as one-hot encoding, is concatenated to the noise provided to the generator as input. The discriminator also receives this instruction, along with the real and synthetic samples. The process is presented in Figure 1 (please note: the real sample’s label needs to be like the one required by the instruction).

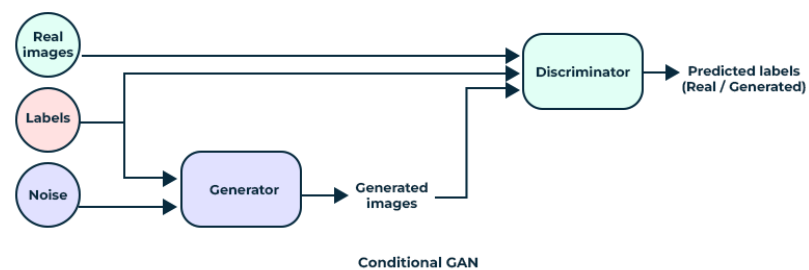


Figure 1: the conditional GAN architecture

- b. Modify your architecture to create a conditional GAN.
- 6) Evaluation and reported results

- a. Provide information on your training process: number of epochs, loss functions, learning rates, optimizers, etc. Produce graphs of losses over time. Remember: in GANs, you will not necessarily see a steady trend throughout training. Fluctuations are quite possible.
- b. After training is complete, use your model to generate a synthetic dataset equal in size to your training set. For the cGAN, create a dataset whose requested label ratios are identical to those of the original training set.
- c. Produce visualizations comparing real vs. synthetic feature distributions (e.g., histograms, correlation matrices). Analyze the results, and elaborate on the meaning of the results (is the synthetic data similar? Problems in the generation process, etc.)
- d. You will evaluate the performance of your models using two common metrics: *detection* and *efficacy*.
- e. The goal of the detection metric is to see whether your synthetic data is similar to the real one. To accomplish this, you will train a Random Forest model on a training set which 50% and 50% synthetic, then evaluate it on a test set with the same proportions. The requested process is as follows:
 - i. Split the original training set (the 80% you used earlier) and synthetic data into four folds.
 - ii. Train the Random Forest on three folds of each data type, then evaluate it on a test set consisting of the two remaining folds. Repeat the process four times, each time with a different test set.
 - iii. Report the average AUC of the Random Forest model. Note: in this case, low performance is good. It means that your generated samples are similar. Analyze and elaborate on the results and their possible causes.
- f. The goal of efficacy is to determine whether your synthetic data is a useful substitute for the real data. The requested process is as follows:
 - i. Train a Random Forest model on the original training set, then evaluate it on the test set (also from the original data). Obtain the AUC score of the model.
 - ii. Train a Random Forest model on the synthetic data, then evaluate it on the test set from the original dataset. Obtain the AUC score of the model.
 - iii. Divide the AUC score of the synthetic data by the AUC score of the original data. Report the results. Note: here, we aim for a high score (with 1 as the maximum), as it indicates that the synthetic data is a good substitute for the real data. Analyze and elaborate on the results and their possible causes.

7) Generating discrete features: gradient flow and representation

- a. The generator you built above must produce categorical features (such as workclass, education, or occupation), but selecting a category with argmax over a softmax is not differentiable. How you represent and back-propagate through categorical outputs is therefore a modeling decision in its own right. Unless stated otherwise, the experiments in this section use your standard (unconditional) GAN with a single random seed; a reduced number of training epochs is acceptable as long as the relevant trends are visible.

- i. Before writing any code, explain in your report why feeding argmax-decoded one-hot vectors to the discriminator prevents the generator from learning. Refer explicitly to gradient flow.
- ii. Implement two differentiable strategies for generating categorical features, and train a GAN with each: (i) the generator outputs, per categorical feature, a softmax distribution that is passed directly to the discriminator; (ii) the generator uses the Gumbel-Softmax reparameterization with a straight-through estimator, so the discriminator receives (near-)discrete samples while gradients still flow back to the generator. For each, describe what happens in the forward and in the backward pass.
- iii. Quantify how committed each strategy's categorical outputs are: report, per categorical feature, the average entropy of the generated soft distributions before decoding, and compare it to the entropy implied by the real category frequencies. A generator that emits near-uniform distributions and relies on argmax is not really generating discrete structure.
- iv. Relate the two strategies to your observed training behavior (loss curves, stability) and to the fidelity of the resulting synthetic data. State which representation you would keep and why, grounding the answer in the evidence you collected rather than in general claims.

8) Mode collapse: induce, diagnose, and mitigate

- a. Mode collapse – the generator producing a narrow set of outputs regardless of its input noise – is the characteristic failure of GAN training. In this section you will produce it deliberately, measure it, and attempt to repair it, using your standard GAN with a single seed and short training runs.
 - i. Define a quantitative collapse indicator for the Adult data and justify it. Acceptable choices include the coverage of distinct categorical-value combinations relative to the real data, or the ratio between synthetic and real per-feature variance (or range) for the continuous features. State precisely how it is computed.
 - ii. Deliberately drive your GAN toward mode collapse – for example by strongly unbalancing the discriminator-to-generator update ratio, or by sharply reducing the generator's capacity – and use your indicator to show that collapse occurred. Include the loss curves and describe what they reveal.
 - iii. Choose one mitigation technique (for example minibatch discrimination, feature matching, a minibatch-standard-deviation layer, or an explicit diversity penalty). Before running it, predict in writing how your collapse indicator and your loss curves should change if the technique works.
 - iv. Apply the mitigation and report whether the indicator recovers. Reconcile the outcome with your prediction, and explain – in terms of what the technique does to the generator or the discriminator – why it did or did not help.

9) Open-ended contribution

- a. Propose and test one non-trivial modification to the architecture or the training procedure of your GAN or cGAN. The idea may come from class, from the sections above, or from you; it must go beyond hyperparameter tuning. This part is graded on the quality of your reasoning and the honesty of your analysis, not on whether the modification improves results.
 - i. State the modification and motivate it from first principles: which limitation of the standard setup are you targeting, and why should your change address it? Write this before running the experiment.
 - ii. Make an explicit, falsifiable prediction – name the metric or the aspect of the training curves you expect to change, and in which direction.
 - iii. Run the experiment (base model, single seed) and present the result alongside the unmodified baseline.
 - iv. Reconcile the result with your prediction. If the modification did not help, diagnose why; a well-argued negative result earns full credit, while an unsupported claim of success does not.